

Project A — 面向对象和垃圾回收

目录

1. 概述	2
2. 分数说明	4
3. 文法概览	5
4. 面向对象特性	6
4.1. open class 与类的继承	6
4.1.1. Object 类	6
4.1.2. super 关键字	6
4.2. 接口 interface 和抽象类 abstract class	7
4.3. 成员函数的覆盖定义和继承冲突	7
4.4. 子类型关系和类型转换	8
4.4.1. Any 类型	8
4.4.2. Nothing 类型	9
4.4.3. 子类型关系 <:	9
4.4.4. is 和 as 操作符	9
4.5. 动态派遣	9
4.6. 额外内置函数	9
5. 内存管理和垃圾回收	11
5.1. 内置函数 gc()	11
5.2. Memory 接口	11
6. 作用域、错误处理、初始化顺序和其他说明	13
7. 输入输出	14
8. 测试用例保证	16
9. 提交	17
9.1. 提交格式	17
9.1.1. 说明	17
9.2. 实验报告要求	17
9.3. 代码提示	18
9.4. 学术诚信 Academic Integrity	18
10. 附录	19
10.1. 错误代码	19
10.1.1. 静态错误代码	19
10.1.2. 动态错误代码	20
10.2. Bonus 满分至少需要支持的程序	21
10.3. Tokens	22
10.4. Grammar	22

1. 概述

在该项目中，你将在 lab 3 实现的类和对象的基础上，进一步支持接口、继承（子类型）、多态（动态派遣）等面向对象编程特性，并实现一个基于 tracing 的垃圾回收器。

⚠ 注意

该项目中**依然需要实现静态类型检查**。同样地，输入也会带上所有的类型标注。

另外，课程项目的框架代码新增了 `cjcj.memory_abstraction` 包（即 `src/memory_abstraction/` 目录）。该包中的文件将在评测时被替换为评测机上的对应文件，其实现方式也可能更改，所以请**不要**依赖该包中的任何非 `public` 的具体实现（`sealed` 隐含 `public`，所以你可以依赖标注了 `sealed` 的接口定义），而是基于其中提供的接口进行编程。

💡 提示

子类型系统与 Hindley-Milner 类型推断不能共存，这是因为泛型约束 + 完整的子类型系统会使得 HM 推断算法直接变得图灵不可判定，在子类型 + 泛型场景下通常使用 local type inference¹。

选择这个 Project，你可能需要完全放弃 Project B（泛型与类型推断）的代码。如果你因为某些原因需要同时选择两个 Project，你需要将你的代码仓库分叉成两个互不影响版本，每个版本分别实现一个 Project 的要求。

¹这部分内容在 12 月 10 日的课上讲过。

🌟 Bonus

你可以参照 Project B 的语法来支持泛型（不过不要实现 `Option` 相关功能，避免影响 Project A 的评测），然后实现 `local type inference`，在实验报告中明确说明你完成了 `bonus`，并说明你的设计与实现思路。

与 Hindley-Milner 类型系统不同，使用 `local type inference` 时往往要求函数体中不能隐式地引入泛型形参。因此，未实例化（没有指明泛型实参）的泛型函数不能作为一等公民使用，要作为一等公民使用必须使用尖括号指明泛型实参，例如 `let f = funcName<ConcreteType>`，而不能直接使用 `let f = funcName`，从而保证函数类型中不会引入额外的泛型形参，函数体中也不会隐式地引入泛型形参。

考虑到工作量原因，该 `bonus` 不要求支持第二操作数为泛型相关类型（泛型类、实例化的泛型类、类型变元）的 `is` 和 `as` 操作符²。另外，该 `bonus` 中函数的参数类型和泛型形参也必须全部给出（即和仓颉的情况一致），不需要推断。

至少需要支持 小节 10.2 中的程序才能拿到 `bonus` 满分。

考虑到部分同学同时选择两个 Project，本就会实现泛型，为保证公平性，只实现泛型的部分和无继承情况下的类型推断（即仅达到 Project B 的“基本要求”），而不支持存在继承时的类型推断将**不得分**。

²这意味着什么呢？你可以结合 12 月 24 日的课程内容，思考一下该 `bonus` 下泛型可以如何选型。

2. 分数说明

Project 由 4 部分测试要求构成：

面向对象特性 (60%)

你需要完成面向对象特性的实现，包括类和接口的定义、继承、子类型关系、动态派遣等。

内存管理和垃圾回收 (30%)

你需要实现基于 tracing 的垃圾回收机制，并使用 `cjcj.memory_abstraction` 包中的 `Memory` 接口和 `memory` 变量进行模拟的内存管理。

综合测试 (10%)

综合测试将同时测试面向对象特性和垃圾回收机制，即在运行包含面向对象特性的程序时，正确地进行内存管理和垃圾回收。

Bonus (10%)

文档中被标记为 Bonus 的内容可以让你额外获得 10% 的分数奖励。Bonus 内容没有测试点，你可以在实验报告中附上实现思路、自制的测试用例和运行结果截图来证明你完成了它们。

提示

如果你从文档第一页读到这里，你已经阅读过了本项目中唯一的 Bonus。

3. 文法概览

本次实验新增加了或修改了下列文法：

(程序)	$P ::= tld^* md tld^*$
(程序入口声明)	$md ::= \dots$
(顶层声明)	$tld ::= vd \mid fd \mid cd \mid ifd$
(表达式块)	$blk ::= \dots$
(声明或表达式)	$expr ::= vd \mid fd \mid e$
(变量声明)	$vd ::= \dots$
(函数声明)	$fd ::= fm^* \text{ func } x (pd_1, pd_2, \dots, pd_n) [blk]$
(函数修饰符)	$fm ::= \text{open} \mid \text{override}$
(函数形参定义)	$pd ::= x : t$
(子类型声明)	$sub ::= <: C_0(\& C_i)^*$
(类声明)	$cd ::= cm^* \text{ class } C[sub] \{ cmd^* \}$
(类修饰符)	$cm ::= \text{abstract} \mid \text{open}$
(类成员声明)	$cmd ::= vd \mid fd \mid init$
(类构造器)	$init ::= \text{init} (pd^*) blk$
(接口声明)	$ifd ::= \text{interface } I[sub] \{ fd^* \}$
(自定义类型)	$A, B, C, I \in \dots$
(变量)	$f, g, x, y \in \dots$
(表达式)	$e ::= \text{super} \mid \text{this} \mid e \text{ is } t \mid e \text{ as } t \mid \dots$
(类型)	$t ::= A \mid \text{Any} \mid \text{Bool} \mid \text{Int64} \mid \text{String} \mid \text{Unit} \mid \text{Nothing} \\ \mid (t_1, t_2, \dots, t_n) \rightarrow t_{ret} \mid \text{Option } <t> \mid ?t$

注：上述文法使用 EBNF 范式，其中

- **abc** 表示终结符号，即源码中的字符串 `abc`；
- `NL` 表示换行符；
- `[...]` 表示可选；
- `(...)*` 表示重复零或多次；
- 为表达简便易懂，上述定义中的若干列表直接使用了省略号 `...` 表示可以特定分隔符重复零或多次的部分，而没有写成 EBNF 的形式。这些列表包括表达式块的参数列表、函数定义的形参列表、函数调用的实参列表、函数类型的参数类型列表等。

提示

子类型声明必须至少包含一个上界类型，或者整体不写，因而没有写成 `...` 的形式。

严格的 BNF 文法参见 小节 10.4。具体程序的 AST 结构请使用实验框架中提供的 `ast` 命令打印查看。

4. 面向对象特性

njucj 为了支持面向对象编程，引入了类 `class` 和接口 `interface`，且支持子类型、动态派遣等特性，并添加了若干修饰符，包括 `abstract`、`open` 和 `override`。

4.1. `open class` 与类的继承

在 njucj 中，类默认是不能被继承的。如果希望一个类可以被继承，需要在 `class` 前使用 `open` 修饰符进行标注。

一个类只能继承一个父类，但可以实现多个父接口。类的继承和类对接口的实现均使用子类型声明的语法 `<: SuperType1 & SuperType2 & ...` 表示。如果某个 `SuperTypei` 不是 `open` 的类或接口，则静态检查时应该在 `TypeNode` 处报错 `SUPER_TYPE_NOT_EXTENSIBLE`。

如果类继承了某个 `open` 的父类，这个父类必须直接写在 `<:` 之后，即 `SuperType1` 的位置，否则在静态检查时应当在子类型声明中的对应 `TypeNode` 处报错 `MISPLACED_SUPERCLASS`。如果 `SuperType1` 已经是一个 `open` 类了，而后续的 `SuperTypei` 也是 `open` 类，同样在静态检查时在 `SuperTypei` 对应的 `TypeNode` 处报错 `MISPLACED_SUPERCLASS`。

⚠ 注意

子类的成员变量不能和父类的成员变量同名，也不能和成员函数同名，否则在子类定义中的 `VarDecl`（注意不是后一处定义，子类定义可以写在父类定义之前）处报错 `DUPLICATED_DEF`。

4.1.1. `Object` 类

在 njucj 中，所有类（除了 `Object` 本身）最终都继承自内置的顶层类 `Object`。如果一个类没有显式地继承某个父类，那么它隐式地继承自 `Object` 类。`Object` 类应当视为通过如下方式定义的类：

```
class Object {}
```

4.1.2. `super` 关键字

在子类中可以使用 `super` 关键字来**静态地**调用父类的成员函数实现，例如 `super.funcName(args...)`；或者使用 `super` 关键字访问在父类中定义的成员变量，例如 `super.f3`。`super` 只能用于成员函数中，且只能调用被调成员函数在祖先类中最近的实现。如果在成员函数外使用 `super`，报错 `THIS_SUPER_OUTSIDE_CLASS`。如果父类中不存在该成员，和其他成员不存在情况一样，报错 `UNDEFINED_VAR`。

在子类的构造器中必须在第一个表达式的位置**首先**使用 `super(...)` 的形式来调用父类的构造器实现，以完成父类部分的初始化，除非父类的构造器无参，此时如果没有显示调用 `super()`，则默认在子类的构造器（子类省略构造器时在子类的默认构造器）开始时先调用父类构造器。如果子类构造器中未首先调用父类构造器（甚至无子类构造器），且父类构造器带参，则在子类 `ClassDecl` 上报错 `SUPER_NOT_INITIALIZED`；如果在类的成员函数或构造器中调用了 `super(...)`，但不在构造器的首个表达式的位置，则报错 `MISPLACED_SUPER_INIT_CALL`（对于根本不在类成员函数或构造函数中调用的 `super(...)`，报错 `THIS_SUPER_OUTSIDE_CLASS`）。父类（包括抽象父类）

³效果和使用 `this` 访问没有区别，只是不能通过 `super` 访问子类中定义的成员变量。

构造函数调用完成时应当动态检查父类成员是否全部初始化，否则在父类 `ClassDecl` 上报错 `MEMBER_NOT_INITIALIZED_AFTER_INIT`。

提示

`super` 只能调用父类的成员函数实现和构造函数，不能调用接口成员函数的默认实现。

4.2. 接口 `interface` 和抽象类 `abstract class`

在 njucj 中，接口使用 `interface` 关键字定义。接口中的成员函数通常不带有函数体，也可以带函数体（此时该成员函数称为带有默认实现的接口成员函数）。接口不能包含构造器和成员变量（已由前端保证）。

接口可以继承其他接口，也使用使用子类型声明的语法 `<: SuperType1 & SuperType2 & ...` 表示。如果某个 `SuperTypei` 不是 `open` 的类或接口，则静态检查时应该在 `TypeNode` 处报错 `SUPER_TYPE_NOT_EXTENSIBLE`；如果是 `open` 的类，则报错 `MISPLACED_SUPERCLASS`。

在类定义的 `class` 关键字前添加 `abstract` 修饰符可以定义抽象类。`abstract` 隐含 `open` 的语义，所以可以不写 `open`。抽象类可以包含成员变量、成员函数和构造器。抽象类中的成员函数可以带有函数体，也可以不带函数体（即抽象成员函数）。抽象类不能被实例化，否则在 `CallExpr` 处报错 `INIT_ABSTRACT_CLASS`，该检查先于函数参数个数和类型的检查。

抽象类的非抽象子类 and 接口的非抽象实现类必须实现所有继承自父类/接口的抽象成员函数，否则在静态检查时应当在类定义的 `ClassDecl` 处报错 `UNIMPLEMENTED_ABSTRACT_MEMBER`。

4.3. 成员函数的覆盖定义和继承冲突

在 njucj 中，如果一个成员函数与父类或父接口中的某个成员函数同名，两者的参数列表（参数个数和类型）均**完全相同**，且前者的返回类型是后者返回类型的子类型，则称前者覆盖定义了后者。此时可以在子类中的该成员函数定义的 `func` 关键字前使用 `override` 修饰符进行标注（也可以省略）。

提示

如果一个成员函数与父类或父接口的多个成员函数同名，则该成员函数需要**同时对所有这些成员函数**构成覆盖定义。

如果成员函数使用了 `override` 修饰符，或者与其所在类继承的父类/接口中的某个成员函数同名，却没有覆盖定义，则在静态检查时在该成员函数的 `FuncDecl` 处报错 `NOT_OVERRIDING`。

非抽象类的成员函数和抽象类中带有函数体的成员函数默认是不能被覆盖定义的。如果希望这两种成员函数可以被覆盖定义，需要在成员函数前使用 `open` 修饰符进行标注。如果一个成员函数尝试覆盖定义了父类/接口中的某个没有使用 `open` 修饰符标注的成员函数，则在静态检查时在子类中的该成员函数的 `FuncDecl` 处报错 `CANNOT_OVERRIDE`。

相反地，抽象类中不带函数体的抽象成员函数和接口中的成员函数（不论是否带函数体）默认都是 `open` 的。

⚠ 注意

open 不会传递。如果一个成员函数使用了 `open` 修饰符进行标注, 且在子类中被覆盖定义, 子类中的该成员函数默认依然不能被再次覆盖定义, 除非子类中的该成员函数也使用了 `open` 修饰符进行标注。

如果一个子类/接口同时继承/实现的父类/接口中（一个父类和若干父接口之间, 或多个父接口之间）存在若干同名的成员函数, 则需要保证这些父类/接口中的同名成员函数的参数列表一致, 否则⁴在静态检查时在该子类的 `ClassDecl` 或子接口的 `InterfaceDecl` 处报错 `INHERITED_FUNC_TYPE_CONFLICT`。对于返回值, 因为任意两个类型总有公共子类型（最坏情况是 `Nothing`）, 所以不会因类型冲突出错⁵。

如果一个（抽象/非抽象）子类/接口继承/实现的父类/接口中存在若干同名成员函数, 这些同名成员函数的参数列表一致, 且其中至少两个带有函数体, 则在该类/接口中必须覆盖定义该成员函数（对于接口和抽象类, 相应的函数可以不带函数体, 也可以带有函数体）, 否则在静态检查时在该子类的 `ClassDecl` 或子接口的 `InterfaceDecl` 上报错 `INHERITED_FUNC_MULTIPLE_IMPLEMENTATION`。

💡 提示

如果一个非抽象子类继承/实现的父类/接口中存在若干同名成员函数, 这些同名成员函数的参数列表一致, 且其中只有一个带有函数体, 但这个带函数体的成员函数的返回类型不是其他同名成员函数返回类型的公共子类型, 且子类/接口中未覆盖定义该成员函数, 则视为子类未实现所有抽象成员函数, 在静态检查时在该子类的 `ClassDecl` 处报错 `UNIMPLEMENTED_ABSTRACT_MEMBER`。

上述情况如果发生在抽象子类或子接口上, 则不应报错, 视为该抽象子类或子接口定义了一个无实现的（抽象）成员函数, 其返回类型为对应父类/接口中成员函数返回类型的公共子类型。

4.4. 子类型关系和类型转换

4.4.1. Any 类型

由于 `njucj` 不像仓颉那样支持为类以外的类型实现接口, 因此引入了顶类型（而不是接口）`Any`, 它是所有其他类型超类型。`Any` 类型可以在需要使用类型的位置显示写出。

💡 提示

`Any` 的类型节点会被语法分析解析为 `RefType` 而不是 `PrimitiveType`。

⁴参数列表不一致时实际上构成了重载, 而本项目不支持重载, 因此需要报错。

⁵子类/接口成员函数的返回类型应当是所有父类/接口中对应成员函数返回类型的子类型, 而 `Nothing` 的存在保证了两个类型总有公共子类型, 进而保证这样的成员函数总是可实现的。关于这种情况, 可以参考 小节 7 样例 2。

4.4.2. **Nothing** 类型

Nothing 类型即底类型，为没有任何值的类型，它是所有类型的子类型。与 lab 3 不同的是，在本项目中，**Nothing** 类型可以显示写出，构成一个 `PrimitiveType` 类型节点。

4.4.3. 子类型关系 `<`:

在 njucj 中，子类型关系 `<`: 包括以下几种情况：

1. 继承类的子类型关系：如果类 `A` 继承自类 `B`，则 `A <: B`，即 `A` 是 `B` 的子类型。
2. 实现接口的子类型关系：如果类 `A` 实现了接口 `I`，则 `A <: I`。
3. 函数类型的子类型关系：函数类型对其参数类型逆变，对其返回类型协变，即对于函数类型 $(T_1, T_2, \dots, T_n) \rightarrow R$ 和 $(U_1, U_2, \dots, U_n) \rightarrow S$ ，如果
 - `U1 <: T1`,
 - `U2 <: T2`,
 - ...,
 - `Un <: Tn`

且 `R <: S`，则 $(T_1, T_2, \dots, T_n) \rightarrow R <: (U_1, U_2, \dots, U_n) \rightarrow S$ 。

4. 永远成立的子类型关系：

1. 一个类型 `T` 永远是自身的子类型，即 `T <: T`;
2. **Nothing** 类型是任意类型 `T` 的子类型，即 `Nothing <: T`;
3. 任意类型 `T` 永远是 **Any** 类型的子类型，即 `T <: Any`。
4. 任意 `class` 定义的类类型 `C` 永远是 **Object** 类型的子类型，即 `C <: Object`，这是因为 **Object** 类是所有类的父类。

4.4.4. **is** 和 **as** 操作符

可以使用 **is** 操作符检查子类型关系。对于表达式 `e is T`，当 `e` 的运行时类型是 `T` 的子类型时，`e is T` 的值为 `true`；否则为 `false`。

子类型的值自然是父类型的值，赋值给父类型的变量时不需要进行类型转换。

在 njucj 中，类型转换必须使用 **as** 操作符显示进行。表达式 `e as T` 的类型为 `Option<T>`。当 `e` 的运行时类型是 `T` 的子类型时，`e as T` 的值为 `Option<T>.Some(e)`；否则为 `Option<T>.None`。

4.5. 动态派遣

对于在继承链上曾经被 **open** 过的成员函数（包括抽象成员函数和接口成员函数（无论是否有默认实现）），调用该成员函数时应当进行动态派遣（子类通过 **super** 关键字调用父类成员函数除外），即根据运行时对象的实际类型来决定调用哪个版本的成员函数。

对于其他的成员函数，实质上保证了只会存在一份实现，也就不需要进行动态派遣。不过，如果你不追求性能的话，也可以对所有成员函数都进行动态派遣；或者如果你想实现静态派遣的话，可以考虑在类型检查阶段修改 AST，甚至构建字节码。

4.6. 额外内置函数

在 lab 3 的基础上，该项目的面向对象部分增加了两个内置函数：

- `func panic(msg: String): Nothing` 调用该函数时应当在 `CallExpr` 处报运行时错误 `RUNTIME_PANIC`，并附上 `msg` 的内容作为错误信息。

- `func refEq(a: Object, b: Object): Bool` 该函数用于比较两个对象引用是否指向同一个对象实例。如果 `a` 和 `b` 指向同一个对象实例，则返回 `true`；否则返回 `false`。如果你实现了垃圾回收，可以安全地复用对象的 `ObjectID` 来实现该函数。

5. 内存管理和垃圾回收

在该项目中，你需要实现基于 tracing 的垃圾回收机制，并**请不要**使用引用计数，**不要**自动进行垃圾回收，只在调用内置函数 `gc()` 时进行垃圾回收。具体的垃圾回收机制请参考[课程讲义](#)。

本项目提供了一部分新的程序，即 `cjcj.memory_abstraction` 包。该包用于提供一系列对内存的抽象，以验证你的内存管理策略。你需要在分配和回收对象时进行一定的操作，以模拟进行堆内存分配和回收的操作（因为如果你直接把对象丢弃的话，会自动利用仓颉本身的垃圾回收机制回收，而这就无法测试了）。

⚠ 注意

`cjcj.memory_abstraction` 包的内容在评测时会被替换为评测机上的对应文件，其实现方式也可能更改，所以请**不要**依赖该包中的任何非 `public` 的具体实现，而是基于其中提供的 `ObjectID` 类、`Memory` 接口和 `memory` 全局变量进行编程。

5.1. 内置函数 `gc()`

在 lab 3 的基础上，该项目的垃圾回收部分增加了内置函数 `func gc(): Unit`，调用该函数时应当触发一次垃圾回收。调用该内置函数时，应当先调用 `memory.onGCBegin()`，再进行垃圾回收，然后调用 `memory.onGCEnd()`。

5.2. `Memory` 接口

`cjcj.memory_abstraction` 包中提供了 `Memory` 接口和 `Memory` 类型的 `memory` 变量。你需要在运行时使用 `memory` 变量来记录对象的分配、移动、回收等操作。`Memory` 是对堆内存空间的一种抽象，该接口定义如下：

```
sealed interface Memory {
    func allocate(object: Object, start: UInt64, size: UInt64): ObjectID
    func move(id: ObjectID, newStart: UInt64): Unit
    func deallocate(id: ObjectID): Unit
    func deallocateExcept(ids: Collection<ObjectID>): Unit
    func onGCBegin(): Unit
    func onGCEnd(): Unit
    static prop maxAddress: UInt64
}
```

💡 提示

这是仓颉代码，不是 njucj 代码。

`Memory` 将堆内存空间分为 `maxAddress` 个单元，每个单元可以是一个 `Int64`、`String`、`Bool`、函数类型值或对象引用。简便起见，只有类对象的实例成员变量需要占用 `Memory` 所代表的堆内存空间。你不需要将虚函数表（如果你的实现中有的话）等元数据存储在 `Memory` 中。初始状态下，`memory` 中的所有内存单元均为未分配状态。我们保证在两次 `gc()` 调用之间，不会有超过 `maxAddress / 2` 个内存单元被分配，因此你可以相对自由地选择使用何种内存分配策略（只要每次分配的空间）。

对象分配时, 请先在仓颉层面上分配好对象的空间, 然后调用 `memory.allocate(...)` 记录该次分配, 你需要传入在仓颉层面上创建的 `class` 的对象 (你最终一定会使用某种 `class` 来为对象分配空间, 因为这是除了 `std.core.LibC.malloc` 等 C 风格内存分配以外, 唯一一种在仓颉中动态分配空间的方式), 以及分配的内存起始地址 `start` 和大小 `size`, 然后该方法将返回一个 `ObjectID`, 之后移动或删除该对象时将会用到。你需要自行确保 `[start, start + size)` 不与其他对象占用的空间重叠, 否则程序将会报错并中止。这里的大小 `size` 应该等于对象的成员变量个数 (包括本身的和继承来的), 而**不要**包含多余部分。如果你的内存分配策略涉及到分配固定大小的空间, 只要让多余部分的空间保持未分配状态即可。如果你希望将 `ObjectID` 放入之前传入的对象中, 你可以使用 `ObjectID` 的默认构造函数构造一个非法的 `ObjectID` 作为临时占位符, 在调用 `memory.allocate(...)` 后再将返回的 `ObjectID` 赋值给对象中的对应成员变量, 或者你也可以使用 `Option`。我们保证每次调用 `memory.allocate(...)` 时返回的 `ObjectID` 都是唯一的。

对象移动时, 请调用 `memory.move(...)` 记录该次移动, 你需要传入之前 `memory.allocate(...)` 返回的 `ObjectID`, 以及新的内存起始地址 `newStart`。你需要自行确保 `[newStart, newStart + size)` 不与其他对象占用的空间重叠 (可以和该对象原先的空间存在重叠), 否则程序将会报错并中止。对象移动不会修改对象对应的 `ObjectID`。

对象释放时, 请调用 `memory.deallocate(id)` 记录对 `id` 对应对象的释放, 或者调用 `memory.deallocateExcept(ids)` 记录对除 `ids` 以外的所有对象的释放。你需要传入之前 `memory.allocate(...)` 返回的 `ObjectID`。释放后, 该对象占用的内存空间可以被重新分配。

进行垃圾回收前后应当分别调用 `memory.onGCBegin()` 和 `memory.onGCEnd()`, 以便我们在评测时进行相应的检查。提供的代码中没有附上具体的检查内容, 但你仍然需要调用这两个方法。

对于 `Memory` 相关的检查, 我们保证只会检查 `cjcj.memory_abstraction` 包中的内容, 不会检查你在调用 `memory.allocate(...)` 时传入 `Object` 的具体内容 (但可能检查对象的地址)。

6. 作用域、错误处理、初始化顺序和其他说明

本项目的错误处理原则与 lab 3 保持一致。

在 lab 3 中没有对成员函数中对成员变量的访问方式具体描述，这里补充如下：

1. 可以通过 `this` 访问其他成员变量和成员函数；
2. 可以直接使用成员变量和成员函数的名字进行访问，等价于通过 `this` 访问，但当成员函数内存在同名局部变量或函数时，优先访问成员函数内的局部变量或函数，这可以近似视为静态作用域嵌套的情况。

鉴于 lab 3 写得不太清楚，我们对其的评测也会相对宽容。

对象（通过类或接口类型的变量）、`Option` 类型和 `Option` 值的成员访问和作用域规则与 lab 3 基本一致。需要额外说明的是，类的继承不会导致完全的作用域嵌套，子类可以直接访问父类的（实例）成员变量，并且子类的成员函数不可以和父类的成员变量和成员函数同名（报错 `DUPLICATED_DEF`）；子类的成员函数只在形成覆盖时和能够和父类成员函数同名（否则报错 `NOT_OVERRIDING`），不能和父类的成员变量同名（报错 `DUPLICATED_DEF`）。上述报错需要在子类中的定义节点处报错。

由于 `njucj` 没有静态成员变量和成员函数，类和接口类型本身不可进行成员访问。

本次项目中不再定义完整的顶层定义初始化顺序，请你自己考虑应该按照怎样的顺序初始化。你依然需要保证按从上到下的顺序初始化全局变量；由于类和接口会作为类型出现在变量、函数甚至其他类和接口的定义中，建议你在进行类型检查之前，先声明好类和接口的类型。另外，由于类和接口的定义之间存在依赖关系，且先写的类和接口定义中可以继承/实现之后定义的和接口，你可能需要使用拓扑排序的方法来确定类和接口的成员处理顺序（主要是静态检查顺序）。我们也会在测试时对报错的优先级进行一定的宽容处理，因为初始化的顺序不同可能导致不同的报错。

不必担心修饰符重复出现，我们已经在语法分析阶段报错处理了这种情况。

7. 输入输出

在本项目中，类型检查和运行同样将会分为两个命令。`check` 只进行类型检查，`run` 则先进行类型检查，再运行程序。

样例 1

执行：`cjcj run`

标准输入：

```
interface ToString {
    func toString(): String
}

class StringPair <: ToString {
    let x: String
    let y: String
    init(x: String, y: String) {
        this.x = x
        this.y = y
    }

    func toString(): String {
        "(" + this.x + "," + this.y + ")"
    }
}

class StringBuilder <: ToString {
    var s: String = ""

    func append(x: ToString): Unit {
        s = s + x.toString()
    }

    func appendString(str: String): Unit {
        s = s + str
    }

    func toString(): String {
        s
    }
}

main(): Unit {
    let p: StringPair = StringPair("234", "Hello")
    let builder: StringBuilder = StringBuilder()
    builder.append(p)
    builder.appendString(" World")
    println(builder.toString())
}
```

标准输出：

```
(234, Hello) World  
()
```

样例 2

执行: `cjcj check`

标准输入:

```
interface A {  
    func f(): Int64 {  
        6  
    }  
}  
  
interface B {  
    func f(): String {  
        "Hello"  
    }  
}  
  
class C <: A & B {  
}  
  
main(): Unit {  
    println(0)  
}
```

标准错误输出:

```
Error at line 13: [INHERITED_FUNC_MULTIPLE_IMPLEMENTATION]: member func 'C.f' has  
multiple implementations 'A.f' and 'B.f'
```

备注: 如果想让这个程序通过检查, 需要在类 `C` 中覆盖定义 `f` 函数, 并且返回类型为 `String` 和 `Int64` 的子类型——也就只有一个: `Nothing`, 这里也只能使用 `panic(msg)` 函数调用表达式来构造符合该类型要求的表达式。

8. 测试用例保证

保证不包含任何具有语法错误的内容。

保证输入一定包含一个程序入口 `main`（事实上已由语法保证）。

保证 `main` 无参数。

保证 `main` 内含有一个或多个表达式或变量/函数定义，即 `main` 的表达式块不为空。

保证不出现未定义的基本类型，即所有显示写出的基本类型一定是 `Int64`、`Bool`、`Unit`、`Nothing` 之一（字符串类型 `String` 和顶类型 `Any` 属于引用类型），不会出现其他仓颉中定义的基本类型⁶。

保证出现已定义的类型时，其类型实参的个数正确，即只有 `Option` 类型会带有一个类型实参，其他类型不带有类型实参。

保证所有可以标注类型的地方均已标注类型。

保证全局变量可以按从上到下的顺序初始化，而不会在过程中引用尚未初始化的全局变量。

⁶即不会在代码中出现 `Int8`、`Int16`、`Int32`、`IntNative`、`UInt8`、`UInt16`、`UInt32`、`UInt64`、`UIntNative`、`Float16`、`Float32`、`Float64`、`Rune` 类型。

9. 提交

提交方式 在智慧南雍平台 (<https://lms.nju.edu.cn>) 提交实验代码和实验报告

截止日期 1/21, 23:59

9.1. 提交格式

将你的代码文件打包成 `projectA_[YOUR_ID]_[YOUR_NAME].zip`, 例如, `projectA_123456789_张三.zip`, 文件树如下

```
projectA_[YOUR_ID]_[YOUR_NAME].zip/  
├── report.pdf # 实验报告  
├── cjpm.toml  # 仓颉项目文件  
├── cjpm.lock  # 仓颉项目文件  
├── src/       # 源代码文件夹  
│   ├── main.cj  
│   ├── ....  
│   └── 你的代码文件
```

9.1.1. 说明

`projectA_[YOUR_ID]_[YOUR_NAME].zip` 文件夹的根目录应该直接包含 `report.pdf`, `cjpm.toml`, `src/` 文件夹。请不要嵌套额外的文件夹, 更不要缺少打包的文件。 **请不要将 `.git`, `target` 等文件夹打包在内。**

在先前的实验中, 出现有同学的压缩包格式错误的情况。为了自动评测的方便, 推荐使用 linux 下的 `zip` 命令打包, 例如:

```
zip -r projectA_123456789_张三.zip report.pdf cjpm.toml cjpm.lock src/
```

提交格式出现错误, 可能导致最终分数出现一定的折扣。

另外, 建议同学们不要在 Windows 上使用 Bandizip 压缩, 因为这款软件不会自动将 Windows 下的文件路径分隔符 `\` 转换为 Unix 风格的 `/`, 从而导致评测机解压后文件夹结构失效。

9.2. 实验报告要求

实验报告命名为 `report.pdf` 并放置在打包的文件夹的根目录。该文件应该是 3~10 页的 PDF。

报告内容应包含以下内容:

1. **你完成的 bonus:** 如果你完成了 bonus, 请在报告中明确说明你完成了哪些 bonus 内容, 并简要描述你的设计与实现思路。请在最前面说明, 以便助教快速找到。
2. **代码结构与设计:** 简要描述你采用的代码结构、设计思路。你无需描述实验框架已经给出的内容。你如何处理继承和多态? 实现了怎样的内存分配和垃圾回收机制? 如何封装和抽象, 从而保持代码整洁, 在实验结束前仍然可以被你阅读? 你的代码有没有什么亮点? 你可以简要描述你认为比较重要或有趣的设计细节。
3. **遇到的问题与解决方案:** 简要描述你在实验过程中遇到的主要问题, 以及你是如何解决这些问题的。如果没有遇到问题, 可以简要描述你认为的实验难度, 和可以改进的地方。

4. **已知 Bug (可选)** : 如果你的代码中存在你已知但未解决的 Bug, 可以简要描述。

9.3. 代码提示

- 创建一个本地的 Git 仓库来跟踪你的更改是一个不错的选择。你也可以将你的代码上传到 GitHub 等平台的私有仓库。保持提交记录的整洁有利于在代码变得复杂时跟踪你的每一行代码是何时编写、为何编写。
- 适当的编写注释有利于你在一段时间后仍能快速理解你编写的代码。对于复杂的代码, 注释也有助于理清自己的思路。
- 在提交代码前, 你可以执行 `cjfmt -d .` 来格式化代码。好的代码风格能方便你和助教的阅读。使用合理的、适当的变量名, 避免过长的函数和代码块。
- 在提交代码前, 请务必确认你的代码可以通过 `cjpm build` 编译。

9.4. 学术诚信 Academic Integrity

学术诚信是所有从事学术活动的学生和学者最基本的职业道德底线, 本课程将不遗余力的维护学术诚信规范, 违反这一底线的行为将不会被容忍。

作业完成的原则: 署你名字的工作必须是你个人的贡献。在完成作业的过程中, 允许讨论, 前提是讨论的所有参与者均处于同等完成度。但关键想法的执行、以及作业文本的写作必须独立完成, 并在报告中致谢 (acknowledge) 所有参与讨论的人。不允许其他任何形式的合作——尤其是与已经完成作业的同学“讨论”。

本课程将对剽窃行为采取零容忍的态度。在完成作业过程中, 对他人工作 (出版物、互联网资料、其他人的作业等) 直接的文本抄袭和对关键思想、关键元素的抄袭, 按照 [ACM Policy on Plagiarism](#) 的解释, 都将视为剽窃。剽窃者成绩将被取消。如果发现互相抄袭行为, 抄袭和被抄袭双方的成绩都将被取消。因此请主动防止自己的作业被他人抄袭。

学术诚信影响学生个人的品行, 也关乎整个教育系统的正常运转。为了一点分数而做出学术不端的行为, 不仅使自己沦为一个欺骗者, 也使他人的诚实努力失去意义。让我们一起努力维护一个诚信的环境。

10. 附录

10.1. 错误代码

10.1.1. 静态错误代码

新增/有修改的静态错误代码：

FUNC_MISSING_BODY 除了抽象类和接口的成员函数外，函数定义中缺少函数体，在 `FuncDecl` 处报错。

SUPER_TYPE_NOT_EXTENSIBLE 子类或接口定义的继承列表中存在不可继承的类型，在该类型对应的 `TypeNode` 处报错。

MISPLACED_SUPERCLASS 子类定义的继承列表中存在不在列表首位的类类型，在该类类型对应的 `TypeNode` 处报错。

UNIMPLEMENTED_ABSTRACT_MEMBER 子类未覆盖定义继承自父类/接口的某些抽象成员函数，在该子类的 `ClassDecl` 处报错。

NOT_OVERRIDING 子类成员函数使用了 `override` 修饰符或者与父类/接口的成员函数同名，但没有覆盖定义任何父类/接口的成员函数，在该成员函数的 `FuncDecl` 处报错。

CANNOT_OVERRIDE 子类成员函数尝试覆盖定义了父类/接口中没有使用 `open` 修饰符标注的成员函数，在该成员函数的 `FuncDecl` 处报错。

INHERITED_FUNC_TYPE_CONFLICT 子类/子接口继承/实现的多个父类/接口中存在同名成员函数，但这些成员函数的参数列表不一致，在该子类的 `ClassDecl` 或子接口的 `InterfaceDecl` 处报错。

INHERITED_FUNC_MULTIPLE_IMPLEMENTATION 子类/子接口继承/实现的多个父类/接口中存在同名成员函数，这些成员函数的参数列表一致，且其中至少两个带有函数体，但子类/子接口没有覆盖定义该成员函数，在该子类的 `ClassDecl` 或子接口的 `InterfaceDecl` 处报错。

INIT_ABSTRACT_CLASS 试图创建抽象类的实例，在 `CallExpr` 处报错。

SUPER_NOT_INITIALIZED 子类构造器中未首先调用父类构造器，且父类构造器带参，在子类 `ClassDecl` 处报错。

MISPLACED_SUPER_INIT_CALL 在类的成员函数或构造器中尝试使用 `super(...)` 调用父类构造函数，但不在构造器的首个表达式的位置，在 `CallExpr` 处报错。

与 lab 3 相同的静态错误代码：

ADD_TYPE_MISMATCH 加法两侧类型不受支持，在 `BinaryExpr` 处报错。

SUB_TYPE_MISMATCH 减法两侧类型不受支持，在 `BinaryExpr` 处报错。

MUL_TYPE_MISMATCH 乘法两侧类型不受支持，在 `BinaryExpr` 处报错。

DIV_TYPE_MISMATCH 除法两侧类型不受支持，在 `BinaryExpr` 处报错。

MOD_TYPE_MISMATCH 取余两侧类型不受支持，在 `BinaryExpr` 处报错。

EXP_TYPE_MISMATCH 幂运算两侧类型不受支持，在 `BinaryExpr` 处报错。

CMP_TYPE_MISMATCH 比较运算两侧的类型不受支持，在 `BinaryExpr` 处报错。

EQ_TYPE_MISMATCH 比较两个不一样的类型，在 `BinaryExpr` 处报错。

NEQ_TYPE_MISMATCH 比较两个不一样的类型，在 `BinaryExpr` 处报错。

AND_TYPE_MISMATCH 在计算中遇到逻辑与的任一操作数不是 `Bool`，在不是 `Bool` 的操作数处报错。

OR_TYPE_MISMATCH 在计算中遇到逻辑或的任一操作数不是 `Bool`，在不是 `Bool` 的操作数处报错。

NOT_TYPE_MISMATCH 逻辑非的操作数不是 `Bool`，在 `UnaryExpr` 处报错。

NEG_TYPE_MISMATCH 一元取负的操作数不是 `Int64`，在 `UnaryExpr` 处报错。

IF_TYPE_MISMATCH if 条件表达式的结果不是 Bool，在 if 的条件表达式处报错；或者 if 的结果被使用，但两个分支的类型没有公共父类型，在 IfExpr 处报错。

WHILE_TYPE_MISMATCH while 条件表达式的结果不是 Bool，在 while 的条件表达式处报错。

BREAK_OUTSIDE_LOOP 在非循环体内使用 break，在 JumpExpr 处报错。

CONTINUE_OUTSIDE_LOOP 在非循环体内使用 continue，在 JumpExpr 处报错。

ASSIGN_IMMUT_DECL 试图给带有初始化的 let 变量、函数（包括带有初始化表达式的 let 成员变量、成员函数）、类等不可变实体赋值，在 AssignExpr 处报错。

ASSIGN_TYPE_MISMATCH 试图给变量赋值成不同的类型，在 AssignExpr 处报错。

UNDEFINED_VAR 试图使用未定义的变量、函数（包括成员变量、成员函数）或类型，在 RefExpr、MemberAccess 或 RefType 处报错。

DUPLICATED_DEF 试图在同一静态作用域内定义同名变量、函数或类型，在 VarDecl、FuncDecl 或 ClassDecl 处报错。

DEF_TYPE_MISMATCH 变量定义时，赋值的类型与声明的类型不匹配，在 VarDecl 处报错。

GLOBAL_NO_INITIALIZER 全局变量定义时缺少初始化表达式，在 VarDecl 处报错。

FUNC_MISSING_RETURN_TYPE 函数定义中省略了返回类型，在 FuncDecl 处报错。不过由于已经保证了所有可以标注类型的地方都标注了类型，因此这个错误实际上不会出现。

CALLEE_NOT_FUNCTION 试图调用一个既不是类（对象创建）也不是函数（函数调用）的值，在 CallExpr 处报错。

CALL_ARG_COUNT_MISMATCH 函数调用（包括对象创建）时传入的参数数量与函数定义不符，在 CallExpr 处报错。

CALL_ARG_TYPE_MISMATCH 函数调用（包括对象创建）时实参类型与形参类型不匹配，在类型不匹配的实参表达式处报错。

FUNC_USE_MUTABLE_NONLOCAL 试图访问可变非局部变量，在 RefExpr 处报错。

FUNC_RETURN_TYPE_MISMATCH 函数返回值类型与声明的返回类型不匹配，在 FuncDecl 处报错。

THIS_SUPER_OUTSIDE_CLASS 在类的方法外使用了 this 或 super 关键字，在 ThisSuperExpr 处报错。

10.1.2. 动态错误代码

新增的动态错误代码：

RUNTIME_PANIC 调用内置函数 panic(msg) 时，在 CallExpr 处报错。

与 lab 3 相同的动态错误代码：

ADD_OVERFLOW 整数加法结果超出 Int64 表示范围，在 BinaryExpr 处报错。

SUB_OVERFLOW 整数减法结果超出 Int64 表示范围，在 BinaryExpr 处报错。

MUL_OVERFLOW 整数乘法结果超出 Int64 表示范围，在 BinaryExpr 处报错。

DIV_BY_ZERO 整数除法的除数为 0，在 BinaryExpr 处报错。

MOD_BY_ZERO 取余运算的除数为 0，在 BinaryExpr 处报错。

EXP_NEGATIVE_POWER 指数为负数的幂运算不被支持，在 BinaryExpr 处报错。

EXP_OVERFLOW 幂运算结果超出 Int64 表示范围，在 BinaryExpr 处报错。

NEG_OVERFLOW 一元取负运算发生溢出，在 UnaryExpr 处报错。

UNINITIALIZED_VAR 试图读取未初始化的变量（包括成员变量）的值，在 RefExpr 或 MemberAccess 处报错。

ASSIGN_IMMUT_VAR 试图给 let 定义的已初始化的不可变变量赋值，在 AssignExpr 处报错。

OPTION_IS_NONE 试图在一个值为 `None` 的 `Option` 上调用 `getOrThrow`, 在 `CallExpr` 处报错。

MEMBER_NOT_INITIALIZED_AFTER_INIT 构造函数执行完毕后, 类的成员变量未被初始化, 在 `ClassDecl` 处报错。

10.2. Bonus 满分至少需要支持的程序

```
class Box<T> {
  var value: T

  init(v: T) {
    value = v
  }
}

abstract class Iterator<T> {
  func next(): ?T
  func map<R>(fn: (T) -> R): Iterator<R> {
    MappedIterator(this, fn)
  }
}

class MappedIterator<T, R> <: Iterator<R> {
  let source: Iterator<T>
  let fn: (T) -> R
  init(source: Iterator<T>, fn: (T) -> R) {
    this.source = source
    this.fn = fn
  }
  func next(): ?R {
    let v = source.next()
    if (v.isSome()) {
      Option<R>.Some(fn(v.getOrThrow()))
    } else {
      Option<R>.None
    }
  }
}

class DummyInt64Iterator <: Iterator<Int64> {
  var current Int64 = 0
  func next(): ?Int64 {
    if (current < 5) {
      let v = current
      current = current + 1
      Option<Int64>.Some(v)
    } else {
      Option<Int64>.None
    }
  }
}

func box<T>(v: T): Box<T> {
  Box(v)
}

main() {
  let it = DummyInt64Iterator().map(box<Int64>)
  while (true) {
```

```
    let v = it.next()
    if (v.isNone()) {
        return 0
    }
    println(v.getOrThrow().value)
}
panic("Unreachable")
}
```

10.3. Tokens

见 [仓颉语言文档 - TokenKind](#)

10.4. Grammar

详见 oopGrammar.md 文件。

CURRENT GRAMMAR

Notes

Look at [Symbols](https://docs.cangjie-lang.cn/en/docs/0.53.13/spec/source_en/Chapter_Appendix_A.html?highlight=syntax#symbols) if you need it.

TRANSLATION UNIT

```
translationUnit
: end* (topLevelObject (end+ topLevelObject?)*)?
  (end+ mainDefinition)? NL* (topLevelObject (end+ topLevelObject?)*)? EOF
;

end
: NL | SEMI
;
```

TOP-LEVEL DEFINITION

```
topLevelObject
: classDefinition
| functionDefinition
| variableDeclaration
| interfaceDefinition
;
```

CLASS DEFINITION

```
classDefinition
: (classModifierList NL*)? CLASS NL* identifier
  (NL* UPPERBOUND NL* superClassOrInterfaces)?
```

```
    NL* classBody
    ;

superClassOrInterfaces
    : superClass (NL* BITAND NL* superInterfaces)?
    | superInterfaces
    ;

classModifierList
    : (classModifier NL*)+
    ;

classModifier
    : ABSTRACT
    | OPEN
    ;

superClass
    : classType
    ;

classType
    : (identifier NL* DOT NL*)* identifier
    ;

superInterfaces
    : interfaceType (NL* BITAND NL* interfaceType )*
    ;

upperBounds
    : type (NL* BITAND NL* type)*
    ;

classBody
    : LCURL end* (classMemberDeclaration (end+ classMemberDeclaration?)*)?
      end* RCURL
    ;

classMemberDeclaration
    : (classInit
    | variableDeclaration
    | functionDefinition
    )? end*
    ;

classInit
    : INIT NL* functionParameters NL* block
    ;

className
    : identifier
    ;
```

INTERFACE DEFINITION

```
interfaceDefinition
  : INTERFACE NL* identifier
  (NL* UPPERBOUND NL* superInterfaces)?
  (NL* interfaceBody)
  ;

interfaceBody
  : LCURL end* interfaceMemberDeclaration* end* RCURL
  ;

interfaceMemberDeclaration
  : (functionDefinition) end*
  ;
```

FUNCTION DEFINITION

```
functionDefinition
  : (functionModifierList NL*)? FUNC
  NL* identifier
  NL* functionParameters
  (NL* COLON NL* type)?
  (NL* block)?
  ;

functionParameters
  : (LPAREN (NL* unnamedParameterList)?
    NL* RPAREN NL*)
  ;

unnamedParameterList
  : unnamedParameter (NL* COMMA NL* unnamedParameter)*
  ;

unnamedParameter
  : (identifier | WILDCARD) NL* COLON NL* type
  ;

functionModifierList
  : (functionModifier NL*)+
  ;

functionModifier
  : OPEN
  | OVERRIDE
  ;
```

VARIABLE DEFINITION

```
variableDeclaration
  : (LET | VAR | CONST) NL* patternsMaybeIrrefutable
    ( (NL* COLON NL* type)? (NL* ASSIGN NL* expression) | (NL* COLON NL*
type) )
  ;

patternsMaybeIrrefutable
  : wildcardPattern
  | varBindingPattern
  ;

varBindingPattern
  : identifier
  ;

wildcardPattern
  : WILDCARD
  ;
```

MAIN ENTRY DEFINITION

```
mainDefinition
  : MAIN
  NL* functionParameters
  (NL* COLON NL* type)?
  NL* block
  ;
```

TYPE

```
// Recheck when need to add Option.
type
  : arrowType
  | prefixType
  | atomicType
  ;

arrowType
  : arrowParameters NL* ARROW NL* type
  ;

arrowParameters
  : LPAREN NL* (type (NL* COMMA NL* type)* NL*)? RPAREN
  ;

prefixType
  : prefixTypeOperator type
  ;
```

```
prefixTypeOperator
: QUEST
;

atomicType
: charLangTypes
| userType
| parenthesizedType
;

charLangTypes
: numericTypes
| RUNE
| BOOLEAN
| Nothing
| UNIT
| THISTYPE
;

numericTypes
: INT8
| INT16
| INT32
| INT64
| INTNATIVE
| UINT8
| UINT16
| UINT32
| UINT64
| UINTNATIVE
| FLOAT16
| FLOAT32
| FLOAT64
;

userType
: identifier ( NL* typeArguments)?
;

parenthesizedType
: LPAREN NL* type NL* RPAREN
;

typeArguments
: LT NL* type (NL* COMMA NL* type)* NL* GT
;
```

EXPRESSION

```
expression
```

```
    : assignmentExpression
    ;

assignmentExpression
    : leftValueExpression NL* ASSIGN NL* logicDisjunctionExpression
    | logicDisjunctionExpression
    ;

leftValueExpression
    : leftValueExpressionWithoutWildCard
    ;

leftValueExpressionWithoutWildCard
    : identifier
    | leftAuxExpression NL* assignableSuffix
    ;

leftAuxExpression
    : identifier
    | type
    | thisSuperExpression
    | leftAuxExpression fieldAccess
    | leftAuxExpression callSuffix
    ;

assignableSuffix
    : fieldAccess
    ;

fieldAccess
    : NL* DOT NL* identifier
    ;

logicDisjunctionExpression
    : logicConjunctionExpression (NL* OR NL* logicConjunctionExpression)*
    ;

logicConjunctionExpression
    : bitwiseDisjunctionExpression (NL* AND NL* bitwiseDisjunctionExpression)*
    ;

bitwiseDisjunctionExpression
    : bitwiseConjunctionExpression (NL* BITOR NL*
bitwiseConjunctionExpression)*
    ;

bitwiseConjunctionExpression
    : equalityComparisonExpression (NL* BITAND NL*
equalityComparisonExpression)*
    ;

equalityComparisonExpression
    : comparisonOrTypeExpression (NL* equalityOperator NL*
```

```
comparisonOrTypeExpression)?  
    ;  
  
comparisonOrTypeExpression  
    : shiftingExpression (NL* comparisonOperator NL* shiftingExpression)?  
    | shiftingExpression (NL* IS NL* type)?  
    | shiftingExpression (NL* AS NL* type)?  
    ;  
  
shiftingExpression  
    : additiveExpression  
    ;  
  
additiveExpression  
    : multiplicativeExpression (additiveOperator NL*  
multiplicativeExpression)*  
    ;  
  
multiplicativeExpression  
    : exponentExpression (NL* multiplicativeOperator NL* exponentExpression)*  
    ;  
  
exponentExpression  
    : prefixUnaryExpression (NL* exponentOperator NL* prefixUnaryExpression)*  
    ;  
  
prefixUnaryExpression  
    : prefixUnaryOperator* postfixExpression  
    ;  
  
postfixExpression  
    : atomicExpression  
    | postfixExpression callSuffix  
    | postfixExpression NL* DOT NL* identifier  
    ;  
  
callSuffix  
    : LPAREN NL* (valueArgument (NL* COMMA NL* valueArgument)* NL*)? RPAREN  
    ;  
  
valueArgument  
    : expression  
    ;  
  
atomicExpression  
    : literalConstant  
    | identifier (NL* typeArguments)?  
    | ifExpression  
    | loopExpression  
    | jumpExpression  
    | thisSuperExpression  
    | parenthesizedExpression  
    ;
```

```
literalConstant
  : IntegerLiteral
  | FloatLiteral
  | booleanLiteral
  | stringLiteral
  | unitLiteral
  ;

booleanLiteral
  : TRUE
  | FALSE
  ;

stringLiteral
  : lineStringLiteral
  ;

lineStringContent
  : LineStrText
  ;

lineStringLiteral
  : QUOTE_OPEN (lineStringContent)* QUOTE_CLOSE
  ;

unitLiteral
  : LPAREN NL* RPAREN
  ;

ifExpression
  : IF NL* LPAREN NL* expression NL* RPAREN NL* block
    (NL* ELSE (NL* ifExpression | NL* block))?
  ;

loopExpression
  : whileExpression
  ;

whileExpression
  : WHILE NL* LPAREN NL* expression NL* RPAREN NL* block
  ;

jumpExpression
  : RETURN (NL* expression)?
  | CONTINUE
  | BREAK
  ;

thisSuperExpression
  : THIS
  | SUPER
  ;
```

```
parenthesizedExpression
  : LPAREN NL* expression NL* RPAREN
  ;

block
  : LCURL expressionOrDeclarations RCURL
  ;

expressionOrDeclarations
  : end* (expressionOrDeclaration (end+ expressionOrDeclaration?)*)?
  ;

expressionOrDeclaration
  : expression
  | varOrfuncDeclaration
  ;

varOrfuncDeclaration
  : functionDefinition
  | variableDeclaration
  ;

assignmentOperator
  : ASSIGN
  ;

equalityOperator
  : NOTEQUAL
  | EQUAL
  ;

comparisonOperator
  : LT
  | GT
  | LE
  | GE
  ;

additiveOperator
  : NL* ADD | SUB
  ;

exponentOperator
  : EXP
  ;

multiplicativeOperator
  : MUL
  | DIV
  | MOD
  ;
```

```
prefixUnaryOperator
: SUB
| NOT
;
```